

Relatório de Auditoria de Segurança — Pelada Pede Mais Uma

Revisão de código, autorização, isolamento por identidade, segredos, XSS, uploads e deploy.

DATA	07 de setembro de 2026
ESCOPO	Aplicação web, API, app móvel, persistência, migrações, servidor self-hosted, Worker, arquivos Docker/Compose/Wrangler, histórico Git e bundles gerados.
COBERTURA	47 arquivos de rota; 98 handlers exportados (incluindo wrappers móveis); 137 commits verificados; 240 testes executados.
RESULTADO	10 achados verificados: 1 crítico, 3 altos e 6 médios. Nenhum achado XSS confirmado.

Nota metodológica

A stack foi detectada antes da busca por falhas. Como o projeto não usa Supabase/RLS nem tabelas multi-tenant, a categoria "banco sem tranca" foi mapeada para o vínculo entre conta e jogador (``player_account_links``) e para filtros derivados da sessão. Gates do React/Expo foram cruzados com os handlers Next.js; todos os handlers foram percorridos. Segredos foram procurados no código atual, deploy, documentação, 137 commits e bundles. XSS foi mapeado para sinks React/React Native, URLs renderizadas, templates de e-mail e renderização de HTML/Markdown.

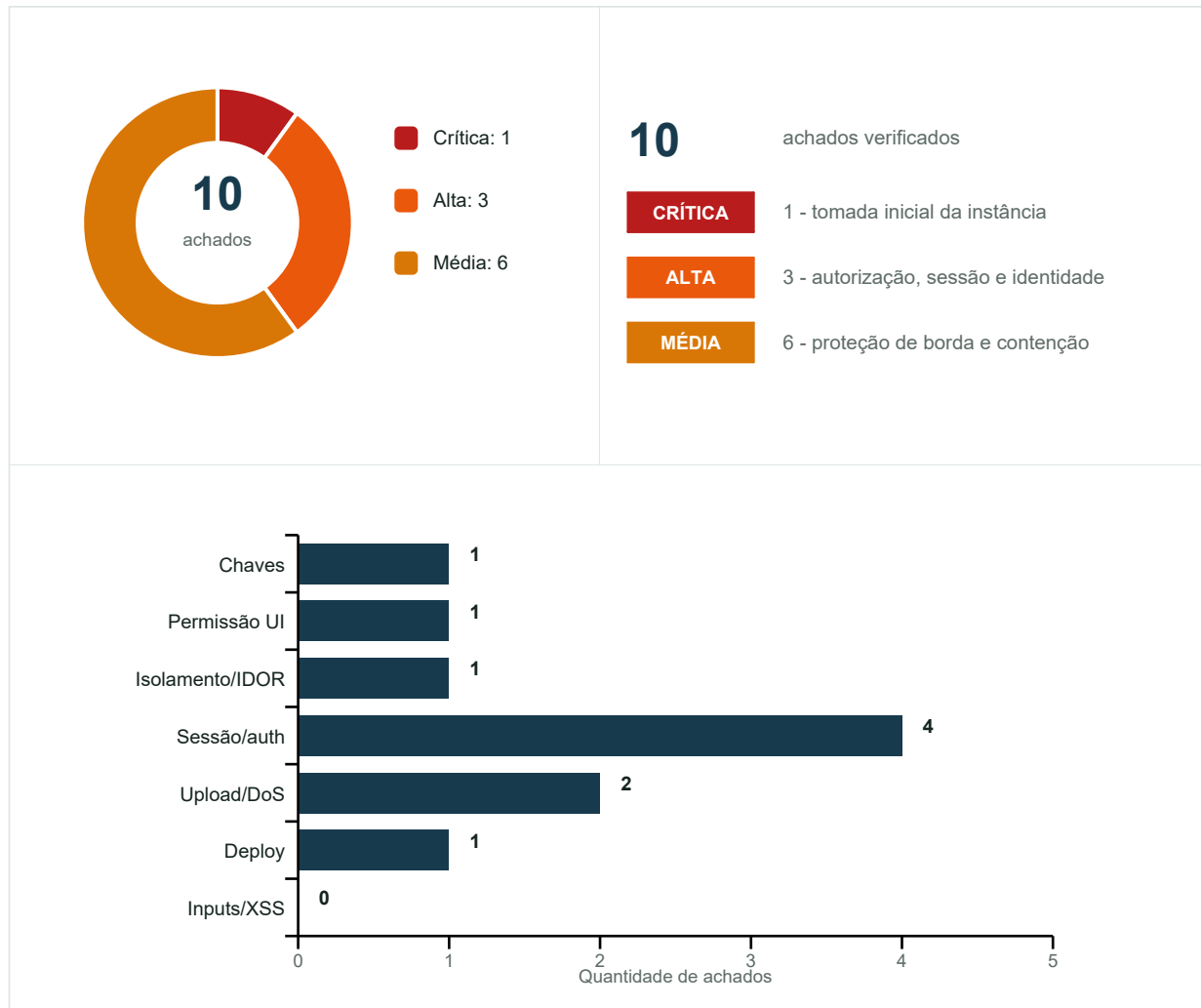
1. Stack e modelo de segurança detectados

Camada	Tecnologia detectada	Mapeamento de auditoria
Linguagem/runtime	TypeScript/JavaScript; Node.js >=22	Handlers, bibliotecas, Worker e servidor self-hosted
Framework web	Next.js 16 via vinext + React 19	App Router em app/api/**/route.ts e gates no React
Persistência	SQLite local / Cloudflare D1; schema Drizzle 0.45	Runtime usa prepared statements D1/SQLite; Drizzle concentra schema/migrações
Autenticação	Cookies opacos web; bearer access/refresh no móvel	Sessões admin/membro, PBKDF2, permissões de moderador e vínculo com jogador
Frontend	React web e Expo/React Native	Gates por accountType/permissions; sinks HTML, URLs, Linking.openURL
Isolamento	Uma pelada por instância; sem tenant_id/RLS	Dados globais da instância; dados pessoais isolados por account_id/playerId
Deploy	Dockerfiles, Docker Compose, Wrangler; sem CI/Helm/Terraform	Segredos/defaults, portas, privilégios de build e sandbox de runtime

Arquivos de CI, Helm e Terraform não existem no repositório; a análise não forçou equivalentes. O arquivo ``.openai/hosting.json`` não está presente. O modelo é single-tenant por implantação, com ``scope_id='instance:1'`` no financeiro; portanto, ausência de ``tenant_id`` nas tabelas globais não foi tratada como falha por si só.

2. Resumo executivo

O risco dominante está no bootstrap administrativo. A credencial conhecida é criada automaticamente, a troca obrigatória é apenas visual e sessões concorrentes sobrevivem à troca. Em paralelo, o primeiro usuário de uma conta comum pode reivindicar um jogador livre sem prova de identidade. Os demais achados concentram-se em controles de borda e contenção: rate limiting, transporte de cookies, hashing de sessões web, uploads e sandbox de contêiner.



A barra Inputs/XSS em zero é resultado de cobertura, não ausência de análise: foram pesquisados sinks HTML/JavaScript, Markdown/HTML, URLs controladas e templates de e-mail. Não há biblioteca de sanitização instalada, mas também não foi localizado ponto que renderize HTML não confiável.

3. Pontos fortes e pontos fracos

Pontos fortes

Autorização sensível no servidor

- Rotas de jogadores, partidas, financeiro, moderadores, configurações, auditoria e releases aplicam `adminRequired` ou `staffRequired` com permissão específica; evidências: app/api/players/route.ts:30-47, app/api/admin/matches/route.ts:14-23 e 48-53, app/api/finance/route.ts:12-26, app/api/moderators/route.ts:8-10 e 30-32.

Posse aplicada em operações pessoais

- Notificações filtram e atualizam por `account\_type` + `account\_id` (app/api/notifications/route.ts:19-31 e 49-61); perfil e ausência derivam `playerId` da sessão (app/api/member-profile/route.ts:10-24 e 27-42; app/api/player-absence/route.ts:6-37); presença ignora playerId do corpo (app/api/matches/route.ts:18-28).

Financeiro pessoal isolado

- Quando o chamador não possui `FINANCE\_MANAGE`, `loadFinance` desvia para `loadPlayerFinance`, que exige `viewer.playerId` e filtra `c.player\_id=?` (lib/finance-service.ts:121-125 e 173-182).

Voto vinculado à identidade autenticada

- O voto usa `account.playerId`, valida participação e possui unicidade por jogador/conta (app/api/career/vote/route.ts:99-139; lib/database.ts:55 e 217).

Tokens sensíveis modernos

- Refresh/access tokens móveis e tokens de redefinição são aleatórios, persistidos como hash, têm expiração/uso único e revogação por reutilização (lib/mobile-auth.ts:20-42 e 64-69; app/api/password-reset/route.ts:63-82; app/api/member-password-reset/route.ts:62-80).

Redução de SQL injection

- As entradas de usuário são passadas por `.bind(...)`; identificadores SQL dinâmicos observados derivam de constantes ou enumerações fechadas, não de texto livre. O adaptador usa prepared statements (server/selfhost-runtime.mjs:11-28).

Superfície XSS bem contida

- Não foram encontrados `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function`, renderizadores Markdown/HTML ou biblioteca de sanitização necessária. React faz escape de texto; URLs de release exigem HTTPS (lib/mobile-release.ts:70-75 e 144-145), branding restringe uploads/HTTPS (lib/instance-config.ts:192-199) e uploads validam assinatura/tipo (app/api/upload/route.ts:19-23).

Segredos operacionais fora do contexto Docker

- `.env` é ignorado no Git e no Docker (linhas 33-36 de .gitignore; linhas 14-16 de .dockerignore). Em 137 commits, o scan por formatos fortes encontrou apenas chaves cliente do Firebase/Google Services esperadas no app móvel; não encontrou chave privada, token backend ou segredo SMTP real. Os bundles `dist/client` e `mobile/dist` não contêm formatos fortes de segredo.

Testes existentes sólidos

- A suíte executada diretamente com Node passou 240/240 testes, incluindo expiração de sessão, autorização de moderador, isolamento financeiro, voto autenticado, reset com revogação e travessia de diretório em upload.

Pontos fracos centrais

- O bootstrap conhecido `admin/admin` combina-se com autorização server-side permissiva durante `must\_change\_password` e com sessões concorrentes não revogadas.
- A identidade de jogador é reivindicada por escolha do primeiro usuário, sem convite, verificação de e-mail ou aprovação.
- Controles de borda (HTTPS/limite de login/limite de corpo/quota) são presumidos, mas não fazem parte dos deploys padrão.
- Sessões web e recipes de contêiner ficam abaixo do nível de proteção já usado em outros componentes do próprio projeto.

4. Achados por categoria

Chaves expostas

Severidade	Arquivo:linha	Descrição
CRÍTICA	lib/database.ts:251-253 app/admin/AdminApp.tsx:47-51 README.md:210	SEC-01 - Credencial administrativa universal é criada e publicada no produto Toda base vazia recebe automaticamente a mesma conta administrativa e a mesma senha. A interface preenche a credencial e a documentação a divulga. Não existe segredo por instância nem validação de startup que recuse a configuração conhecida.

Permissão no navegador

Severidade	Arquivo:linha	Descrição
ALTA	app/admin/AdminApp.tsx:20-22 lib/database.ts:263-271 app/api/administrators/route.ts:3-5	SEC-02 - Troca obrigatória de senha é apenas um gate de frontend A UI impede a navegação normal quando `mustChangePassword` é verdadeiro, porém o servidor continua tratando a sessão como administrador completo. A regra de negócio existe no navegador e não é repetida nos endpoints.

Isolamento/IDOR

Severidade	Arquivo:linha	Descrição
ALTA	app/api/member-auth/route.ts:40-59 app/api/member-players/route.ts:3-7 app/api/member-players/route.ts:10-25 app/api/member-profile/route.ts:27-42	SEC-04 - Cadastro público pode reivindicar qualquer jogador ainda não associado O isolamento por dono é implementado por `player_account_links`, mas a etapa que cria o vínculo aceita qualquer `playerId` livre. Não há prova de posse, convite, código de ativação ou aprovação administrativa.

Sessão/autenticação

Severidade	Arquivo:linha	Descrição
ALTA	app/admin/AdminApp.tsx:59 app/api/auth/route.ts:37-40 app/api/profile/password/route.ts:25-28	SEC-03 - Troca inicial de senha não revoga sessões concorrentes O endpoint de primeiro acesso substitui a senha, mas não invalida as demais sessões web que foram abertas com a credencial padrão. O próprio fluxo de troca posterior demonstra o comportamento esperado ao revogar outras sessões.
MÉDIA	app/api/auth/route.ts:14-19 app/api/member-auth/route.ts:17-25 app/api/mobile/auth/route.ts:13-23 docker-compose.yml:27-28	SEC-05 - Endpoints de login não possuem limitação de tentativas Login administrativo, portal do jogador e login móvel aceitam tentativas ilimitadas. A documentação transfere rate limiting ao proxy, mas o Compose padrão publica diretamente a aplicação e não configura esse proxy.
MÉDIA	app/api/auth/route.ts:5 app/api/member-auth/route.ts:4 docker-compose.yml:27-28 docker-compose.ovm.yml:40-41	SEC-06 - Cookies de sessão não usam o atributo Secure Os cookies têm HttpOnly e SameSite=Strict, mas não `Secure`. Portanto, o navegador pode enviá-los em HTTP quando a origem estiver acessível dessa forma.
MÉDIA	app/api/auth/route.ts:20-29 app/api/member-auth/route.ts:27-34 lib/database.ts:263-265 lib/mobile-auth.ts:65-69	SEC-07 - Tokens de sessão web são persistidos em texto claro Sessões web são bearer tokens reutilizáveis e ficam em texto claro no SQLite/D1, embora tokens móveis e de recuperação sejam armazenados somente como hash.

Upload/DoS

Severidade	Arquivo:linha	Descrição
MÉDIA	app/api/upload/route.ts:13-20	SEC-08 - Limite de upload é aplicado somente depois de carregar o corpo inteiro Quando `Content-Length` é omitido ou usa transferência em chunks, o pré-check recebe zero e o handler materializa todo o corpo em memória antes de descobrir que excede 5 MB.
MÉDIA	app/api/upload/route.ts:7-24 server/selfhost-runtime.mjs:119-134	SEC-09 - Uploads ilimitados geram objetos órfãos sem quota ou limpeza Cada envio cria um objeto permanente antes de o URL ser associado a qualquer jogador. Envios abandonados, substituições de foto e abuso deliberado não são removidos nem contabilizados por usuário.

Deploy/hardening

Severidade	Arquivo:linha	Descrição
MÉDIA	docker-compose.omv.yml:10-15 docker-compose.omv.yml:45-50 Dockerfile.selfhost.omv:7-17 Dockerfile.multiarch:9-21	SEC-10 - Build e runtime OMV desabilitam barreiras do sandbox de contêiner As receitas de produção concedem modo inseguro/privilegiado ao build e removem o filtro seccomp no runtime OMV. `cap_drop: ALL`, usuário não-root e `no-new-privileges` ajudam, mas não restauram essas duas fronteiras.

Inputs/XSS

SEM ACHADOS	Nenhuma falha verificável nesta categoria. Veja os pontos fortes e a metodologia de cobertura.
-------------	--

5. Achados detalhados

CRÍTICA

Chaves expostas

SEC-01

Credencial administrativa universal é criada e publicada no produto

Descrição

Toda base vazia recebe automaticamente a mesma conta administrativa e a mesma senha. A interface preenche a credencial e a documentação a divulga. Não existe segredo por instância nem validação de startup que recuse a configuração conhecida.

Evidência - arquivo por arquivo, linha por linha

lib/database.ts:251-253

```
if(!admin){ const hash=await hashPassword("admin"); ... .bind(crypto.randomUUID(),"admin",hash,1,1,now,now).run(); }
```

app/admin/AdminApp.tsx:47-51

```
const [mode,...,[email,setEmail]=useState('admin'),[password,setPassword]=useState('admin'),...
```

README.md:210

```
No primeiro acesso self-hosted, use `admin` / `admin`.
```

Por que é explorável

Um atacante que alcance uma instalação recém-criada pode autenticar-se como administrador sem adivinhação. O impacto é controle total de contas, jogadores, finanças, partidas, configurações e uploads.

Condições de explorabilidade

Explorável enquanto a instância ainda conserva a credencial inicial; o risco é especialmente alto nas portas HTTP publicadas pelos Compose. Depois da troca, este achado isolado deixa de ser explorável, mas os achados SEC-02 e SEC-03 ampliam a janela.

Correção recomendada

Gerar bootstrap aleatório por instância, exibi-lo uma única vez por canal local/console, exigir configuração explícita segura e bloquear o servidor se a credencial conhecida existir em ambiente não local.

ALTA

Permissão no navegador

SEC-02

Troca obrigatória de senha é apenas um gate de frontend

Descrição

A UI impede a navegação normal quando `mustChangePassword` é verdadeiro, porém o servidor continua tratando a sessão como administrador completo. A regra de negócio existe no navegador e não é repetida nos endpoints.

Evidência - arquivo por arquivo, linha por linha

app/admin/AdminApp.tsx:20-22

```
if(admin.mustChangePassword)return <FirstAccess ... />;
```

lib/database.ts:263-271

```
currentAdmin(...) seleciona must_change_password, mas não exige must_change_password=0; adminRequired apenas retorna currentAdmin.
```

app/api/administrators/route.ts:3-5

```
GET/POST/PUT de administradores confiam somente em adminRequired(request).
```

Por que é explorável

Após entrar com `admin/admin`, basta chamar as APIs diretamente para cadastrar outro administrador, alterar configurações, gerir contas ou operar o financeiro sem trocar a senha inicial.

Condições de explorabilidade

Requer uma sessão administrativa com `must\_change\_password=1`, obtida normalmente no primeiro acesso ou de uma base restaurada nesse estado.

Correção recomendada

Fazer `adminRequired` recusar contas com troca pendente em todas as rotas, liberando somente `/api/auth` PUT, logout e recuperação; adicionar testes negativos para cada família privilegiada.

ALTA

Sessão/autenticação

SEC-03

Troca inicial de senha não revoga sessões concorrentes

Descrição

O endpoint de primeiro acesso substitui a senha, mas não invalida as demais sessões web que foram abertas com a credencial padrão. O próprio fluxo de troca posterior demonstra o comportamento esperado ao revogar outras sessões.

Evidência - arquivo por arquivo, linha por linha

app/admin/AdminApp.tsx:59

```
FirstAccess envia PUT /api/auth.
```

app/api/auth/route.ts:37-40

```
UPDATE administrators SET email=?,password_hash=?,must_change_password=0,... WHERE id=?
```

app/api/profile/password/route.ts:25-28

```
O fluxo posterior de perfil, em contraste, executa DELETE FROM sessions WHERE administrator_id=? AND id<>?.
```

Por que é explorável

Um atacante pode entrar com a senha padrão antes do responsável. Mesmo após a troca legítima, a sessão do atacante continua válida por até oito horas e mantém poderes administrativos.

Condições de explorabilidade

Requer login concorrente durante a janela de bootstrap e uma sessão ainda não expirada.

Correção recomendada

Executar a troca em transação e apagar todas as sessões web e mobile da conta, opcionalmente mantendo apenas uma nova sessão rotacionada criada após a alteração.

ALTA

Isolamento/IDOR

SEC-04

Cadastro público pode reivindicar qualquer jogador ainda não associado

Descrição

O isolamento por dono é implementado por `player\_account\_links`, mas a etapa que cria o vínculo aceita qualquer `playerId` livre. Não há prova de posse, convite, código de ativação ou aprovação administrativa.

Evidência - arquivo por arquivo, linha por linha

app/api/member-auth/route.ts:40-59

```
PUT /api/member-auth cria conta ativa e sessão sem verificação de e-mail ou convite.
```

app/api/member-players/route.ts:3-7

```
GET /api/member-players lista jogadores ativos sem vínculo.
```

app/api/member-players/route.ts:10-25

```
POST /api/member-players insere o vínculo usando o playerId enviado pelo chamador.
```

app/api/member-profile/route.ts:27-42

```
PUT /api/member-profile passa a editar o jogador derivado desse vínculo.
```

Por que é explorável

Qualquer pessoa pode criar uma conta com um e-mail arbitrário, enumerar os jogadores sem conta e assumir um deles. A partir daí recebe identidade esportiva, pode votar como o jogador, ver cobranças pessoais e editar nome, posição, observações e foto desse perfil.

Condições de explorabilidade

Explorável para jogadores ainda não associados. As restrições UNIQUE evitam dupla associação depois da primeira reivindicação, mas não autenticam quem fez a primeira.

Correção recomendada

Trocar a escolha aberta por convite de uso único emitido por administrador ou por fluxo de solicitação/aprovação; verificar e-mail; impedir listagem pública de candidatos e auditar a aprovação.

MÉDIA

Sessão/autenticação

SEC-05

Endpoints de login não possuem limitação de tentativas

Descrição

Login administrativo, portal do jogador e login móvel aceitam tentativas ilimitadas. A documentação transfere rate limiting ao proxy, mas o Compose padrão publica diretamente a aplicação e não configura esse proxy.

Evidência - arquivo por arquivo, linha por linha

app/api/auth/route.ts:14-19

```
Os três handlers consultam a conta e chamam verifyPassword sem contador, atraso, logout, 429 ou Retry-After.
```

app/api/member-auth/route.ts:17-25

```
ports: - "${HOST_PORT:-3000}:${PORT:-3000}"
```

Por que é explorável

Um atacante remoto pode testar senhas indefinidamente e distribuir tentativas por contas/IPs. O PBKDF2 aumenta o custo, mas não limita a taxa nem protege contra password spraying.

Condições de explorabilidade

Explorável quando não há proxy/WAF externo com limitação efetiva; é a configuração entregue pelos Compose do repositório.

Correção recomendada

Implementar limites por IP e por conta, backoff progressivo, resposta 429 com Retry-After e telemetria; manter uma segunda camada no proxy.

MÉDIA

Sessão/autenticação

SEC-06

Cookies de sessão não usam o atributo Secure

Descrição

Os cookies têm HttpOnly e SameSite=Strict, mas não `Secure`. Portanto, o navegador pode enviá-los em HTTP quando a origem estiver acessível dessa forma.

Evidência - arquivo por arquivo, linha por linha

app/api/auth/route.ts:5

```
`${name}=${value}; HttpOnly; SameSite=Strict; Path=/; Max-Age=${maxAge}`
```

app/api/member-auth/route.ts:4

Os Compose expõem a porta HTTP da aplicação; não há HSTS configurado no código.

Por que é explorável

Em rede hostil, downgrade, acesso direto à porta publicada ou proxy TLS mal configurado, o token de sessão pode atravessar a rede sem criptografia e ser reutilizado.

Condições de explorabilidade

Requer que o usuário acesse a origem por HTTP ou que exista rota HTTP até a aplicação. Um proxy que force HTTPS reduz a exposição, mas o cookie continua sem defesa própria.

Correção recomendada

Adicionar `Secure` em produção, redirecionar HTTP para HTTPS, habilitar HSTS no ponto de entrada e falhar startup quando a URL pública de produção não for HTTPS.

MÉDIA

Sessão/autenticação

SEC-07

Tokens de sessão web são persistidos em texto claro

Descrição

Sessões web são bearer tokens reutilizáveis e ficam em claro no SQLite/D1, embora tokens móveis e de recuperação sejam armazenados somente como hash.

Evidência - arquivo por arquivo, linha por linha

app/api/auth/route.ts:20-29

```
INSERT INTO sessions/member_sessions grava diretamente o UUID entregue no cookie.
```

app/api/member-auth/route.ts:27-34

```
currentAdmin/currentMember comparam `s.id=?` com o valor bruto do cookie.
```

lib/database.ts:263-265

```
O fluxo móvel, em contraste, persiste `hashOpaqueToken(accessToken)` e `hashOpaqueToken(refreshToken)`.
```

Por que é explorável

Leitura indevida do banco, snapshot, backup ou console permite replay imediato das sessões: até oito horas para administrador e até trinta dias para membro.

Condições de explorabilidade

Depende de exposição de leitura da base ou de backup. Não exige quebra do hash de senha.

Correção recomendada

Persistir apenas SHA-256/HMAC do token aleatório, comparar o hash recebido, rotacionar sessões existentes e reduzir/revisar TTLs conforme o risco.

MÉDIA

Upload/DoS

SEC-08

Limite de upload é aplicado somente depois de carregar o corpo inteiro

Descrição

Quando `Content-Length` é omitido ou usa transferência em chunks, o pré-check recebe zero e o handler materializa todo o corpo em memória antes de descobrir que excede 5 MB.

Evidência - arquivo por arquivo, linha por linha

app/api/upload/route.ts:13-20

```
const declaredSize = Number(request.headers.get("content-length") || 0);
```

Por que é explorável

Um membro autenticado pode enviar corpos muito grandes ou várias requisições paralelas para pressionar memória e CPU do processo self-hosted, causando indisponibilidade antes da resposta 413.

Condições de explorabilidade

Requer conta autenticada; SEC-04 torna a criação dessa conta pública. Limites impostos externamente por Cloudflare/proxy reduzem a exposição, mas o servidor Node local não contém limite de streaming próprio.

Correção recomendada

Impor limite no servidor/proxy antes do handler e ler o stream incrementalmente, abortando assim que superar 5 MB; limitar concorrência e testar requisição chunked sem Content-Length.

MÉDIA

Upload/DoS

SEC-09

Uploads ilimitados geram objetos órfãos sem quota ou limpeza

Descrição

Cada envio cria um objeto permanente antes de o URL ser associado a qualquer jogador. Envios abandonados, substituições de foto e abuso deliberado não são removidos nem contabilizados por usuário.

Evidência - arquivo por arquivo, linha por linha

app/api/upload/route.ts:7-24

```
Cada POST autorizado cria `players/${crypto.randomUUID()}` e chama UPLOADS.put.
```

server/selfhost-runtime.mjs:119-134

```
A busca por `UPLOADS.delete` no código de produção não encontrou chamadas; não há vínculo transacional com o perfil nem quota por c
```

Por que é explorável

Uma conta pode repetir uploads válidos de até 5 MB até esgotar o volume local ou elevar custos no R2, afetando disponibilidade e operação.

Condições de explorabilidade

Requer conta autenticada; não há limite de frequência, volume acumulado ou quantidade. A capacidade física do storage é o único freio observado.

Correção recomendada

Aplicar quota e rate limit por conta, registrar dono/estado temporário, confirmar associação em prazo curto e executar coleta de órfãos; remover o objeto anterior quando uma foto for substituída.

MÉDIA

Deploy/hardening

SEC-10

Build e runtime OMV desabilitam barreiras do sandbox de contêiner

Descrição

As receitas de produção concedem modo inseguro/privilegiado ao build e removem o filtro seccomp no runtime OMV. `cap\_drop: ALL`, usuário não-root e `no-new-privileges` ajudam, mas não restauram essas duas fronteiras.

Evidência - arquivo por arquivo, linha por linha

docker-compose.omv.yml:10-15

```
build: ... privileged: true
```

docker-compose.omv.yml:45-50

```
security_opt: - seccomp:unconfined
```

Dockerfile.selfhost.omv:7-17

```
RUN --security=insecure ... npm ci / npm run build
```

Por que é explorável

Código malicioso introduzido no repositório ou cadeia de dependências durante o build recebe uma superfície maior contra o host; no runtime OMV, uma exploração do Node/app dispõe de mais syscalls para pós-exploração.

Condições de explorabilidade

Aplica-se aos builds que habilitam entitlement `security.insecure` e ao Compose OMV. O runtime do Compose padrão não declara `seccomp:unconfined`.

Correção recomendada

Eliminar `privileged` e `--security=insecure`; se a limitação do kernel OMV impedir isso, isolar o builder, documentar a exceção, fixar e verificar artefatos, usar SBOM/assinatura e criar um perfil seccomp mínimo compatível para runtime.

6. Recomendações priorizadas

P1 - Bootstrap seguro e autorização de primeiro acesso

Resolver SEC-01, SEC-02 e SEC-03 em conjunto antes de publicar nova instância: remover `admin/admin`, bloquear APIs durante troca pendente e revogar sessões concorrentes.

P2 - Prova de identidade para vínculo com jogador

Substituir a associação aberta por convite/aprovação com verificação de e-mail (SEC-04). Revisar jogadores atualmente vinculados em instâncias existentes.

P3 - Controles de autenticação e transporte

Aplicar rate limiting nos três logins e `Secure` + HTTPS/HSTS (SEC-05 e SEC-06). Tratar proxy como defesa adicional, não requisito invisível.

P4 - Redução do impacto de vazamento de banco

Migrar sessões web para token hash e revogar registros legados (SEC-07).

P5 - Proteção de upload

Limitar streaming, concorrência, taxa e quota; introduzir ownership/estado temporário e garbage collection (SEC-08 e SEC-09).

P6 - Hardening de build e OMV

Retirar entitlements inseguros e criar perfil seccomp mínimo; reforçar supply chain com digest, assinatura e SBOM (SEC-10).

P7 - Testes de regressão

Adicionar testes específicos para os dez achados. Preservar os 240 testes atuais, que validam várias proteções já existentes.

7. Cobertura e limitações

- Revisão estática de todos os 47 `route.ts` em `app/api`, totalizando 98 exports de método HTTP quando wrappers móveis são contados.
- Cruzamento de gates `accountType`, `role`, `permissions`, `isAdmin` e `can*` do React/Expo com os endpoints chamados.
- Revisão dos helpers de autenticação, sessão, finanças, partidas, notificações, uploads, URLs públicas, e-mail e adapters D1/SQLite.
- Busca por sinks XSS/execução (`dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function`, Markdown/HTML) e por URLs em href/src/Linking.openURL.
- Busca de segredos em fonte, exemplos, Compose/Docker, Wrangler, documentação, 137 commits e bundles. Chaves `google-services.json` foram classificadas como identificadores cliente esperados; a segurança real delas depende de restrições no console Google, fora do código auditado.
- Execução da suíte: 240 testes passaram. `npm test` não iniciou devido a npm global quebrado no host; o comando equivalente `node --test tests/*.test.mjs` foi executado com sucesso.
- Sem teste dinâmico contra ambiente publicado, proxy, Cloudflare, conta Google/Firebase, SMTP, registry ou host OMV. Controles externos não versionados não foram presumidos.
- Worktree já continha alterações em `mobile/dist`; elas não foram modificadas nem usadas como fonte autoritativa, exceto no scan somente leitura de formatos de segredo.

ISSUES PARA O GITHUB

Os 10 achados foram agrupados em 7 issues para evitar duplicação. Cada bloco abaixo está completo e pronto para copiar e colar em Markdown.

Issue 1 de 7

[Segurança] Eliminar credencial admin universal e bloquear APIs até concluir o bootstrap

```
--- ISSUE 1 ---
Título: [Segurança] Eliminar credencial admin universal e bloquear APIs até concluir o bootstrap

Labels sugeridas: security, severidade:crítica

## Descrição

Instalações vazias criam automaticamente `admin/admin`, a interface preenche essa credencial e a documentação a publica. Depois do login, `must_change_password` é respeitado apenas pela UI: `adminRequired` aceita a sessão em todas as APIs. Além disso, a troca inicial não revoga sessões abertas em paralelo.

## Por que é explorável

Quem alcançar uma instância nova pode autenticar-se sem adivinhação, chamar diretamente endpoints administrativos e manter uma sessão por até oito horas mesmo que o responsável troque a senha logo depois.

## Evidência

- `lib/database.ts:251-253`

```ts
if(!admin){ const hash=await hashPassword("admin");
 ...bind(crypto.randomUUID(),"admin",hash,1,1,now,now).run(); }
```

- `app/admin/AdminApp.tsx:22,49,59`

```tsx
if(admin.mustChangePassword)return <FirstAccess ... />;
const [email,setEmail]=useState('admin'),[password,setPassword]=useState('admin');
// FirstAccess envia PUT /api/auth
```

- `lib/database.ts:263-271` - `adminRequired` não exige `must_change_password=0`.
- `app/api/auth/route.ts:37-40` - a troca inicial atualiza a senha sem apagar sessões.

## Impacto

Tomada completa da instância: contas, perfis, partidas, resultados, finanças, configurações e uploads.

## Sugestão de correção

Gerar segredo inicial aleatório por instância ou fluxo de criação local; recusar startup inseguro em produção; fazer `adminRequired` bloquear contas pendentes em toda rota exceto troca/logout/reset; na conclusão, revogar todas as sessões web/mobile e emitir token novo.

## Critérios de aceite

- [ ] Base vazia não contém senha universal nem valor documentado.
- [ ] Produção falha de forma segura se o bootstrap não estiver configurado.
- [ ] Sessão com `must_change_password=1` recebe 403 em toda API privilegiada.
- [ ] Apenas troca inicial, logout e recuperação permanecem acessíveis nesse estado.
- [ ] Troca inicial revoga todas as sessões concorrentes web e mobile.
- [ ] Teste automatizado prova que a sessão antiga deixa de funcionar.
--- FIM ISSUE 1 ---
```

Issue 2 de 7

[Segurança] Exigir convite ou aprovação para associar conta a jogador

--- ISSUE 2 ---

Título: [Segurança] Exigir convite ou aprovação para associar conta a jogador

Labels sugeridas: security, severidade:alta

Descrição

O portal permite cadastro público sem verificação de e-mail, lista jogadores ainda sem conta e aceita o `playerId` escolhido pelo usuário. O vínculo passa a ser a fonte de autorização para voto, perfil, ausência e financeiro.

Por que é explorável

Um atacante cria uma conta com qualquer e-mail, seleciona um jogador livre e assume a identidade antes do titular. O índice UNIQUE protege apenas contra a segunda associação; não autentica a primeira.

Evidência

- `app/api/member-auth/route.ts:40-59` - cria conta ativa e sessão sem convite/verificação.

- `app/api/member-players/route.ts:3-7`

```ts

```
SELECT ... FROM players p LEFT JOIN player_account_links l ON l.player_id=p.id
WHERE ... AND l.player_id IS NULL
```

```

- `app/api/member-players/route.ts:10-25`

```ts

```
const playerId = String(payload.playerId || "");
INSERT INTO player_account_links (...) .bind(playerId, ..., member.id, ...)
```

```

- `app/api/member-profile/route.ts:27-42` - o vínculo permite editar o perfil.

Impacto

Impersonação de jogador, voto indevido, acesso a cobrança pessoal e alteração de dados do perfil.

Sugestão de correção

Usar convite de uso único emitido por administrador ou solicitação pendente aprovada por administrador; verificar e-mail; não expor a lista completa de candidatos a contas não aprovadas.

Critérios de aceite

- [] Cadastro novo não cria vínculo ativo por escolha unilateral.
- [] Convite/solicitação está vinculado a jogador e expira.
- [] Administrador aprova e a ação fica auditada.
- [] E-mail da conta é verificado antes de ativar o vínculo.
- [] Tentativa com `playerId` arbitrário recebe 403.
- [] Testes cobrem corrida de dois solicitantes para o mesmo jogador.

--- FIM ISSUE 2 ---

Issue 3 de 7

[Segurança] Aplicar rate limiting aos três fluxos de login

--- ISSUE 3 ---

Título: [Segurança] Aplicar rate limiting aos três fluxos de login

Labels sugeridas: security, severidade:média

Descrição

Login administrativo, login do jogador e login móvel verificam credenciais sem limite por conta/IP, atraso progressivo, lockout temporário ou resposta 429. O Compose padrão publica a aplicação diretamente.

Por que é explorável

Permite brute force e password spraying contínuos. O PBKDF2 eleva o custo por tentativa, mas não controla volume distribuído.

Evidência

```
- `app/api/auth/route.ts:14-19`  
- `app/api/member-auth/route.ts:17-25`  
- `app/api/mobile/auth/route.ts:13-23`  
- `docker-compose.yml:27-28`
```

```
```yaml
```

```
ports:
```

```
- "${HOST_PORT:-3000}:${PORT:-3000}"
```

```
```
```

Impacto

Comprometimento de contas com senhas fracas/reutilizadas e consumo de CPU por PBKDF2.

Sugestão de correção

Limitar por IP e identificador normalizado, usar backoff progressivo e janela deslizando, responder 429/Retry-After e alertar sobre spraying. Manter limite adicional no proxy.

Critérios de aceite

- [] Todos os três endpoints compartilham política de limite.
- [] Excesso recebe 429 com Retry-After.
- [] Resposta não revela se a conta existe.
- [] Há limite por conta e por origem.
- [] Testes cobrem janela, desbloqueio e IPs distribuídos.

--- FIM ISSUE 3 ---

Issue 4 de 7

[Segurança] Marcar cookies de sessão como Secure e forçar HTTPS

```
--- ISSUE 4 ---
Título: [Segurança] Marcar cookies de sessão como Secure e forçar HTTPS

Labels sugeridas: security, severidade:média

## Descrição

Cookies web usam HttpOnly e SameSite=Strict, mas não `Secure`. Os Compose publicam uma
porta HTTP e o código não aplica HSTS.

## Por que é explorável

Se o usuário acessar HTTP, a sessão pode ser transmitida sem criptografia e reutilizada
por quem observa a rede.

## Evidência

- `app/api/auth/route.ts:5`
- `app/api/member-auth/route.ts:4`

```ts
`${name}=${value}; HttpOnly; SameSite=Strict; Path=/; Max-Age=${maxAge}`
```

- `docker-compose.yml:27-28`; `docker-compose.omv.yml:40-41`.

## Impacto

Sequestro de sessão administrativa ou de jogador em implantação HTTP/downgrade.

## Sugestão de correção

Adicionar `Secure` em produção, forçar HTTPS/HSTS no proxy e rejeitar configuração pública
não HTTPS.

## Critérios de aceite

- [ ] Set-Cookie contém Secure em produção.
- [ ] HTTP redireciona para HTTPS antes de autenticação.
- [ ] HSTS está presente na origem pública.
- [ ] Startup de produção recusa APP_BASE_URL HTTP.
- [ ] Testes verificam os atributos em login e logout.
--- FIM ISSUE 4 ---
```

Issue 5 de 7

[Segurança] Persistir somente hash dos tokens de sessão web

--- ISSUE 5 ---

Título: [Segurança] Persistir somente hash dos tokens de sessão web

Labels sugeridas: security, severidade:média

Descrição

Os UUIDs entregues nos cookies são gravados em claro em `sessions` e `member\_sessions`. Tokens móveis e de reset já usam hash, mostrando uma implementação segura disponível.

Por que é explorável

Quem obtiver leitura do banco ou backup pode copiar o valor e usá-lo como bearer token sem conhecer a senha.

Evidência

- `app/api/auth/route.ts:20-29`; `app/api/member-auth/route.ts:27-34`

```
```ts
INSERT INTO sessionsbind(token, account.id, ...)
INSERT INTO member_sessionsbind(id, account.id, ...)
```
```

- `lib/database.ts:263-265` compara o cookie diretamente a `s.id`.
- `lib/mobile-auth.ts:65-69` usa `hashOpaqueToken` nos tokens móveis.

Impacto

Replay de sessão por até 8 horas (admin) ou 30 dias (membro) após vazamento de banco/backup.

Sugestão de correção

Emitir token aleatório de alta entropia, persistir SHA-256/HMAC, comparar apenas o hash e migrar/revogar sessões legadas.

Critérios de aceite

- [] Banco não contém o bearer token entregue ao cliente.
- [] Lookup autentica pelo hash.
- [] Sessões antigas são revogadas/migradas.
- [] Logs e auditoria nunca registram o token.
- [] Teste comprova que o valor armazenado não autentica diretamente.

--- FIM ISSUE 5 ---

Issue 6 de 7

[Segurança] Tornar uploads limitados por streaming, quota e coleta de órfãos

--- ISSUE 6 ---

Título: [Segurança] Tornar uploads limitados por streaming, quota e coleta de órfãos

Labels sugeridas: security, severidade:média

Descrição

O endpoint confia no Content-Length para o pré-check e, quando ele falta, usa `request.arrayBuffer()` antes de validar 5 MB. Cada upload cria ainda um objeto permanente sem quota, rate limit, dono persistido ou limpeza.

Por que é explorável

Uma conta pode enviar corpo chunked grande para consumir memória e repetir imagens válidas para esgotar storage/custos. SEC-04 facilita obter essa conta.

Evidência

- `app/api/upload/route.ts:13-23`

```
``ts
const declaredSize = Number(request.headers.get("content-length") || 0);
const buffer = await request.arrayBuffer();
if (buffer.byteLength > MAX_FILE_SIZE) return ... 413;
await UPLOADS.put(`${purpose}/${crypto.randomUUID()}.${extension}`, buffer, ...);
``
```

- `server/selfhost-runtime.mjs:119-134` persiste o objeto.

- Não há chamada de produção a `UPLOADS.delete`.

Impacto

Indisponibilidade por memória/concorrência e esgotamento do volume/R2 por objetos órfãos.

Sugestão de correção

Limitar corpo no proxy e por leitura incremental, abortar acima de 5 MB, aplicar rate/quota por conta, registrar dono/estado temporário e coletar objetos não associados.

Critérios de aceite

- [] Upload chunked acima de 5 MB é abortado sem buffer integral.
- [] Há limite de concorrência e taxa por conta/IP.
- [] Quota cumulativa impede esgotamento do storage.
- [] Objeto temporário expira se não for associado.
- [] Troca/remoção de foto coleta o objeto antigo quando seguro.
- [] Métricas alertam sobre volume e rejeições.

--- FIM ISSUE 6 ---

Issue 7 de 7

[Segurança] Remover build privilegiado e seccomp unconfined do deploy OMV

--- ISSUE 7 ---

Título: [Segurança] Remover build privilegiado e seccomp unconfined do deploy OMV

Labels sugeridas: security, severidade:média

Descrição

O Compose OMV marca o build como privilegiado, os Dockerfiles executam etapas com `--security=insecure` e o runtime OMV desabilita seccomp.

Por que é explorável

Código malicioso no repositório/dependências durante build ganha uma fronteira mais fraca contra o host; uma exploração do runtime dispõe de mais syscalls para pós-exploração.

Evidência

- `docker-compose.omv.yml:10-15`

```
```yaml
build:
 privileged: true
```
```

- `docker-compose.omv.yml:45-50`: `seccomp:unconfined`.

- `Dockerfile.selfhost.omv:7-17` e `Dockerfile.multiarch:9-21`: `RUN --security=insecure`.

Impacto

Aumento do impacto de comprometimento de supply chain/build e de exploração do processo em produção OMV.

Sugestão de correção

Remover entitlements inseguros; separar builder; usar artefatos assinados/SBOM; criar perfil seccomp mínimo compatível e preservar usuário não-root, cap_drop e no-new-privileges.

Critérios de aceite

- [] Build padrão funciona sem privileged e security.insecure.
- [] Runtime não usa seccomp:unconfined.
- [] Perfil seccomp mínimo é versionado e testado no OMV alvo.
- [] Imagens são fixadas por digest/assinadas e acompanhadas de SBOM.
- [] Pipeline valida que exceções inseguras não retornaram.

--- FIM ISSUE 7 ---